

■ GenAI RAG Chunking Playground: Codebase Teaching & Architecture Guide

Welcome to the teaching manual for the **Chunking Playground** codebase. This guide is written in plain language to help you explain the project architecture, dependencies, algorithms, and agentic workflows to a learner or manager.

■ Part 1: How to Start with the Codebase (Where to Read First)

When introducing a new student to this repository, guide them in this chronological order:

1. The Core Configuration (The Setup)

- **requirements.txt**: Check what libraries are installed. Explain that this is a Python backend coupled with a web frontend.
- **README.md**: Standard quick-start instructions for local setup and python virtual environments (`.venv`).

2. The File Ingestion Layer (How Documents are Read)

Before chunking text, we must extract it from documents. Have the student look at:

- **txt_reader.py**: Basic plain text extraction.
- **pdf_reader.py**: Extracts text from PDFs using the **pypdf** parser.
- **docx_reader.py**: Extracts text from Word documents using the **python-docx** parser.

3. The API Gateway (The Entry Point)

- **main.py**: The central routing file built with FastAPI. It handles uploads, starts chunk comparisons, simulates search retrieval, serves static frontend files, and handles the PDF generation trigger `/api/download-guide`.

4. The Chunker Algorithms (The Logic)

Once they understand how text is loaded and routed, introduce the chunkers:

- **fixed_chunker.py** -> **recursive_chunker.py** -> **document_chunker.py** -> **semantic_chunker.py** -> **query_aware_chunker.py** -> **metadata_chunker.py** -> **llm_chunker.py** -> **agentic_chunker.py**

■■ Part 2: Python Dependency Breakdown (What We Use & Why)

Here is a list of the core Python packages listed in **requirements.txt** and their specific purpose in the application:

1. fastapi & uvicorn

- **Why**: FastAPI provides a high-performance, asynchronous web server framework to expose endpoints. Uvicorn acts as the lightweight ASGI server to run the app.

2. langchain & langchain-text-splitters

- **Why**: Used to leverage standard document loaders and industry-default splitters like **RecursiveCharacterTextSplitter** to show students how standard RAG pipelines partition text.

3. langgraph

- **Why:** To build stateful, multi-node agent orchestrators. It models the self-routing agentic chunker as an executable graph.

4. sentence-transformers

- **Why:** Generates dense vector embeddings (specifically using the lightweight, local **all-MiniLM-L6-v2** transformer model). We use these embeddings for both semantic similarity chunking and RAG search simulation.

5. pypdf & python-docx

- **Why:** Handles binary parsing for PDF documents and Word (.docx) file uploads.

6. tiktoken

- **Why:** OpenAI's tokenizer wrapper. We use it to estimate the actual token count of each chunk, showing students the difference between character limits and LLM token counts.

7. scikit-learn & numpy

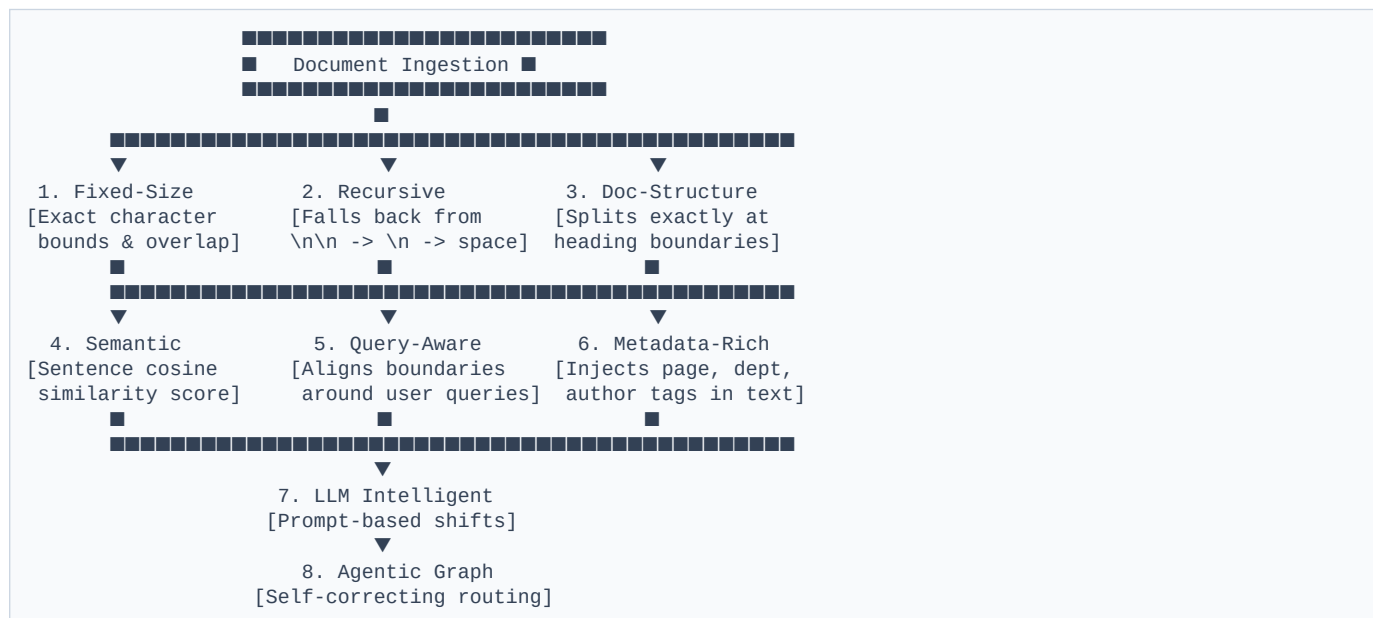
- **Why:** Standard scientific computing libraries. Used to compute **cosine similarity** between sentence vectors (for topic shifts) and query vectors (for search simulation).

8. reportlab

- **Why:** Standard PDF layout and compiler library. Generates the dynamically compiled RAG Chunking Reference Guide and live sandbox session report.

■ Part 3: Deep Dive into the 8 Chunking Strategies

RAG performance is entirely dependent on how documents are segmented. Review these 8 strategies with the learner:



1. Fixed-Size Character Chunking (fixed_chunker.py)

- **How it works:** Slices text at static character counts (e.g. 500 characters) with a sliding overlap window (e.g. 50 characters) to ensure boundary continuity.
- **Pedagogical highlight:** Shows boundary flaws (breaking words or sentence structures in half), illustrating why heuristic chunking is a fallback.

2. Recursive Character Chunking (`recursive_chunker.py`)

- **How it works:** Splits text hierarchically using a sequence of delimiters: double newlines (`\n\n`), single newlines (`\n`), spaces (), and characters ("").
- **Pedagogical highlight:** Keeps paragraphs and sentences intact by only splitting at smaller separators when chunk sizes exceed target constraints.

3. Document-Structure Aware Chunking (`document_chunker.py`)

- **How it works:** Analyzes formatting markers (like Markdown headings `#`, `##` or HTML layouts) and splits content along logical sections.
- **Pedagogical highlight:** Keeps section headers grouped with their corresponding lists or tables, preventing context dilution.

4. Semantic Similarity Chunking (`semantic_chunker.py`)

- **How it works:** Splits documents into sentences, generates embeddings for each sentence, evaluates the cosine similarity between adjacent sentence vectors, and splits the document when similarity falls below a statistical threshold.
- **Pedagogical highlight:** Teaches how vector databases evaluate meaning; chunks adjust dynamically to match shifts in the text's topic.

5. Query-Aware Chunking (`query_aware_chunker.py`)

- **How it works:** Dynamically adjusts chunk boundaries around areas of the text that have high similarity to a specific user question.
- **Pedagogical highlight:** Optimizes retrieval precision by isolating query answers from surrounding irrelevant paragraphs.

6. Metadata-Enhanced Chunking (`metadata_chunker.py`)

- **How it works:** Automatically discovers document tags (department, version, document name) and injects them directly into the text block of each chunk.
- **Pedagogical highlight:** Helps the vector database filter chunks accurately by preserving global context (e.g. department filters) inside micro-segments.

7. LLM-Based Intelligent Chunking (`llm_chunker.py`)

- **How it works:** Employs an LLM to read the document and identify topic shift points, returning chunk boundaries, titles, and explanations.
- **Pedagogical highlight:** Mimics human reading and classification capabilities, but demonstrates latency and API token cost trade-offs.

8. Agentic Workflow Chunking (`agentic_chunker.py`)

- **How it works:** Orchestrates strategy routing and evaluation using a state machine (LangGraph). (See details in Part 4).

■■ Part 4: What is LangGraph & How is it Used Here?

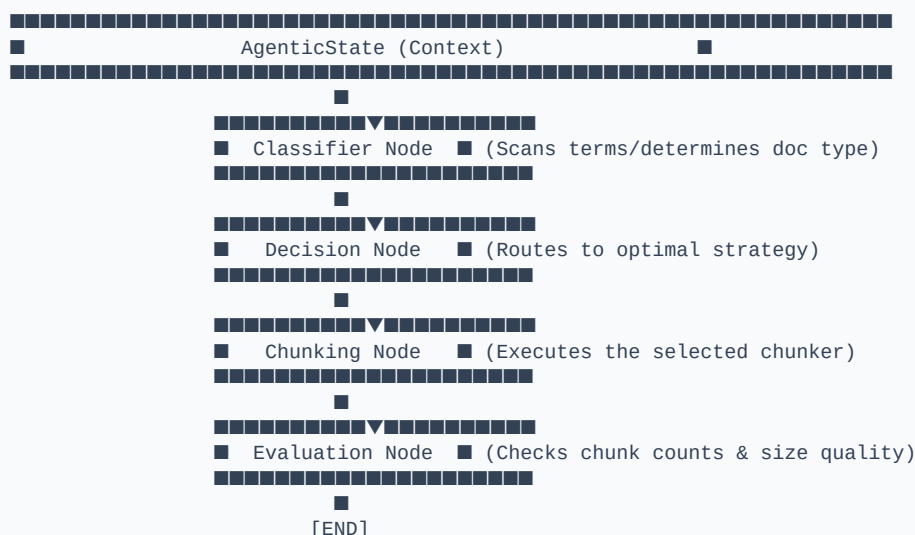
1. Concept of LangGraph

LangGraph is a stateful orchestration framework that extends LangChain. It models agentic workflows as a **State Graph**:

- **State**: A common dictionary (**AgenticState**) passed chronologically through nodes, storing current document text, metrics, and logs.
- **Nodes**: Standard Python functions executing single steps of the pipeline (classification, splitting, evaluation).
- **Edges**: Directed paths connecting the nodes, indicating execution order.
- **Compilation**: Graph compiles into a single runnable object (`compiled_graph = workflow.compile()`) invoked via `.invoke(initial_state)`.

2. Multi-Node Pipeline in the Playground

The agentic loop in `agentic_chunker.py` uses 4 distinct nodes:



1. Classifier Node (classifier_node):

- **Logic**: Scans the input text for keyword patterns. For example, if it detects **q:** or **faq**, it classifies the document as a "Company FAQ" with 95% confidence. If it detects **abstract** and **conclusion**, it classifies it as a "Research Paper" with 94% confidence.

2. Decision Node (decision_node):

- **Logic**: Receives the document type and selects the optimal chunking strategy. FAQ maps to *Query-Aware*, Research Papers map to *Document-Structure*, Policies map to *Semantic Similarity*, and standard text maps to *Recursive*.

3. Chunking Node (chunking_node):

- **Logic**: Calls the respective Python backend service to split the text. If a selected strategy fails, it falls back to the reliable *Recursive Chunker* to prevent application crashes.

4. Evaluation Node (evaluation_node):

- **Logic**: Assesses the generated chunks. It computes standard metrics (such as average chunk size and chunk count) and assigns a quality score (e.g., scoring higher if chunks fall in the healthy 200-800 character window).

3. Why Use LangGraph for Chunking?

- It models how production RAG ingestion pipelines dynamically analyze incoming text documents (rather than forcing a single fixed chunk size onto FAQs, slides, logs, and legal codes alike).

- It provides a visual timeline (the "Thinking Timeline") so students can audit the logical steps of the orchestrator.

■ Part 5: The Sandbox Simulators & Caching System

Explain to the learner how the sandbox handles data and simulates production RAG pipelines:

1. Retrieval Simulation (`main.py -> retrieve_chunks_sync`)

To test chunking quality, the student submits a search query:

1. **Embedding:** The search query is embedded using the `all-MiniLM-L6-v2` transformer model.
2. **Similarity check:** Cosine similarity is computed between the query vector and all chunk vectors.
3. **Metadata Boost:** If a chunk contains metadata matching key terms in the query, it receives a retrieval confidence boost of up to `0.40`.
4. **Metrics Output:**
 - *Precision:* Percentage of top-K results that have a similarity score ≥ 0.40 .
 - *Recall:* Percentage of all relevant chunks in the database successfully retrieved in the top-K.
 - *Mean Similarity:* Average cosine similarity of the top matches.

2. Session Caching Layer (`session_manager.py`)

To prevent re-calculating embeddings and re-running expensive LLM simulation cycles, the backend saves intermediate results as JSON files inside `backend/data/`:

- `metadata.json`: Current document text, file type, and readability metrics.
- `chunks.json`: Generated chunk list, strategy parameters, and execution latency.
- `stats.json`: Benchmarking matrix comparing all strategy counts, sizes, coherence, and processing speeds.
- `retrieval_results.json`: The simulated search query, precision/recall metrics, and matches.

When the PDF Study Guide is compiled, the backend reads these files dynamically and appends a professional **Active Workspace Report** showing the learner's actual experimental outcomes.

■ Summary for Classroom Presentation

Use this quick summary sheet when speaking to the class or manager:

- *Purpose:* Visually teach RAG document segmentation and vector search retrieval before sending data to models.
- *Frontend:* Premium glassmorphic theme (light/dark) built with Vanilla HTML/JS and Chart.js.
- *Backend:* High-performance FastAPI routing, using LangChain splitters, SentenceTransformers embeddings, and ReportLab PDF compilers.
- *Autonomous Orchestrator:* Stateless StateGraph classifier loop modeled via LangGraph.
- *Data Persistence:* Lightweight JSON filesystem cache in the backend `data` directory.